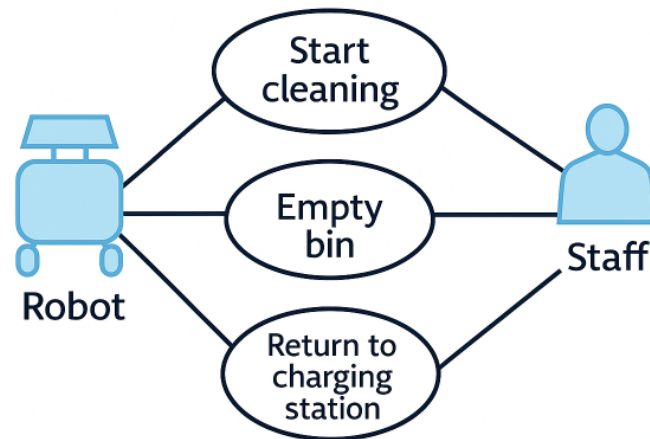

1.

Garbage Cleaning Robot



“Proudly” generate using AI. Other use-cases that would be obvious includes at least ‘avoid obstacles’ and ‘notify of full bin’.

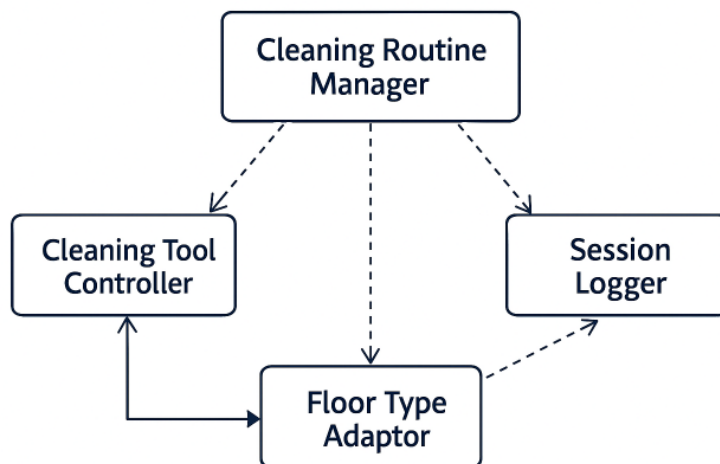
2. Possible modules (not exclusive list!) could be:

- Cleaning monitor
 - Executes the cleaning routine
 - Controls brushing/suction systems
- Navigation & path-planning
 - Plan routes around tables, chairs, people
 - Position tracking during cleaning
- Garbage bin monitoring
 - Detect garbage fill level
 - Trigger a “bin full” event

3. The cleaning module could be divided into something like this:

- Cleaning routine manager (the manager)
 - Cleaning tool controller (On/Off for suction & brush)
 - Floor type adaptor (Adjust cleaning based on floor surface)
 - Cleaning session logger (Provide statistics for user)
-

4.



Note – the AI image generator (yes, I am lazy) only draws approximate visuals and does not understand UML semantics. This is why the arrows in the above picture are not UML-accurate and do not reflect the actual dependencies.

- Cleaning Routine Manager → Cleaning Tool Controller: commands like start/stop tools, set brush speed
- Cleaning Routine Manager → Floor Type Adaptor: requests current floor type; receives change events
- Cleaning Routine Manager → Session Logger: logs lifecycle events (started, paused, error, completed)
- Floor Type Adaptor → Cleaning Tool Controller: applies parameter updates (suction level, brush pressure) based on detected surface
- Floor Type Adaptor → Session Logger: logs floor-type changes or adaptation actions

5. Entities represent data, hardware, or states. We need to identify the entities, why they are needed, and what each module must be able to do with them.

Using the Floor Type Adaptor as an example we may find the following entities:

- FloorType (e.g. “tile”, “carpet”, “linoleum”)
- SensorReading (optical reflectance, vibration, pressure)
- AdaptedToolSettings (output settings depending on floor type)
- FloorChangeEvent (signals changes to the Routine Manager)

What the module must be able to do:

- Interpret raw SensorReading values
 - Determine current FloorType
 - Produce AdaptedToolSettings for Cleaning Tool Controller
-

-
- Notify Routine Manager using FloorChangeEvent

This module is essentially a “translator” between sensor data and cleaning behavior.

6. Rough code-like structure could be as follows:

```
//----- Session Logger Module-----  
class SessionLogger {  
    public void log(String message) {  
        System.out.println("[LOG] " + message);  
    }  
}  
  
//----- Cleaning Tool Controller-----  
class CleaningToolController {  
    public void applyToolSettings(String settings) {  
        System.out.println("CleaningToolController: Applying settings-> " + settings);  
    }  
}  
  
//----- Floor Type Adaptor-----  
class FloorTypeAdaptor {  
    public String detectFloorType() {  
        // Floor type sensing takes place here  
        return "carpet";  
    }  
  
    public String generateToolSettingsForFloor(String floorType) {  
        if (floorType.equals("carpet")) {  
            return "High suction, slow brush speed";  
        } else {  
            return "Normal suction, normal brush speed";  
        }  
    }  
}  
  
//----- Cleaning Routine Manager-----  
class CleaningRoutineManager {  
    private final CleaningToolController toolController;  
    private final FloorTypeAdaptor floorTypeAdaptor;  
    private final SessionLogger logger;  
  
    public CleaningRoutineManager(CleaningToolController toolController,  
                                  FloorTypeAdaptor floorTypeAdaptor,  
                                  SessionLogger logger) {  
        this.toolController = toolController;  
    }  
}
```

```

        this.floorTypeAdaptor = floorTypeAdaptor;
        this.logger = logger;
    }

    public void startCleaningRoutine() {
        logger.log("Cleaning routine started.");

        // Ask floor-type adaptor for the current floor
        String floorType = floorTypeAdaptor.detectFloorType();
        logger.log("Detected floor type: " + floorType);

        // Floor adaptor generates tool settings
        String settings = floorTypeAdaptor.generateToolSettingsForFloor(floorType);

        // Apply these tool settings to the tool controller
        toolController.applyToolSettings(settings);

        logger.log("Cleaning routine active with adapted settings.");
    }
}

//----- MAIN program-----
public class Main {
    public static void main(String[] args) {
        CleaningToolController toolController = new CleaningToolController();
        FloorTypeAdaptor floorTypeAdaptor = new FloorTypeAdaptor();
        SessionLogger logger = new SessionLogger();

        CleaningRoutineManager manager =
            new CleaningRoutineManager(toolController, floorTypeAdaptor, logger);

        manager.startCleaningRoutine();
    }
}

```

7. One suggestion for a bicycle description could be:

- Drivetrain (Pedal → Crank → Chain → GearSystem)
 - GearSystem (Front chainring(s), Rear cassette, Derailleur)
- Wheel (front, rear) with Tire, Rim, Hub, Spokes, Axle, Nut + Bolt
- Brake (front, rear) (simple rim brake abstraction)
- Lights (front, rear) with LightSensor
- Frame (placeholder entity to show structure)

With rough modules described the behavioral highlights could be:

-
- `stepOnPedals(force)` → converts rider effort into speed, modulated by gear ratio
 - `shiftUp()/shiftDown()` → changes ratio (higher = faster but harder)
 - `applyFrontBrake(force)`, `applyRearBrake(force)`
 - `updateAmbientLight(lux)` → lights auto on/off
 - `removeWheel(position)`, `fixPuncture(position)`
-

8. Focusing on the core classes needed to model the bicycle at a reasonable teaching level has here been interpreted as looking at

- Bicycle
- GearSystem
- Wheel
- Brake
- Light

With this we may get something like the following rough example code structure:

```
//----- GEAR SYSTEM-----  
class GearSystem {  
    private int gear = 1;  
  
    public void shiftUp() {  
        if (gear < 7) {  
            gear++;  
            System.out.println("Shifting up: now in gear " + gear);  
        } else {  
            System.out.println("Already in highest gear.");  
        }  
    }  
  
    public void shiftDown() {  
        if (gear > 1) {  
            gear--;  
            System.out.println("Shifting down: now in gear " + gear);  
        } else {  
            System.out.println("Already in lowest gear.");  
        }  
    }  
  
    public int getGear() {  
        return gear;  
    }  
}
```

```
//----- WHEEL-----
class Wheel {
    private final String position; // "front" or "rear"
    private boolean mounted = true;

    public Wheel(String position) {
        this.position = position;
    }

    public void remove() {
        if (!mounted) {
            System.out.println(position + " wheel is already off.");
        } else {
            mounted = false;
            System.out.println(position + " wheel off.");
        }
    }

    public void mount() {
        if (mounted) {
            System.out.println(position + " wheel is already mounted.");
        } else {
            mounted = true;
            System.out.println(position + " wheel mounted.");
        }
    }
}
```

```
//----- BRAKE-----
abstract class Brake {
    private final String position;

    protected Brake(String position) {
        this.position = position;
    }

    public void apply() {
        System.out.println(position + " brake applied.");
    }
}

class FrontBrake extends Brake {
    public FrontBrake() { super("Front"); }
}
```

```
class RearBrake extends Brake {
```

```
    public RearBrake() { super("Rear"); }  
}
```

```
//----- LIGHT-----  
class Light {  
    private final String position;  
    private boolean on = false;  
  
    public Light(String position) {  
        this.position = position;  
    }  
  
    public void switchOn() {  
        if (!on) {  
            on = true;  
            System.out.println(position + " light on.");  
        }  
    }  
  
    public void switchOff() {  
        if (on) {  
            on = false;  
            System.out.println(position + " light off.");  
        }  
    }  
}
```

```
9. //----- BICYCLE-----  
class Bicycle {  
    private final GearSystem gears = new GearSystem();  
    private final Wheel frontWheel = new Wheel("Front");  
    private final Wheel rearWheel = new Wheel("Rear");  
    private final FrontBrake frontBrake = new FrontBrake();  
    private final RearBrake rearBrake = new RearBrake();  
    private final Light frontLight = new Light("Front");  
    private final Light rearLight = new Light("Rear");  
  
    //--- Actions defined in the original description---  
  
    public void stepOnPedals() {  
        System.out.println("Pedaling forward.");  
    }  
  
    public void shiftUp() {  
        gears.shiftUp();  
    }  
}
```

```
    public void shiftDown() {
        gears.shiftDown();
    }

    public void applyFrontBrake() {
        frontBrake.apply();
    }

    public void applyRearBrake() {
        rearBrake.apply();
    }

    public void switchLightsOn() {
        frontLight.switchOn();
        rearLight.switchOn();
    }

    public void switchLightsOff() {
        frontLight.switchOff();
        rearLight.switchOff();
    }

    public void removeFrontWheel() {
        frontWheel.remove();
    }

    public void removeRearWheel() {
        rearWheel.remove();
    }

    public void mountFrontWheel() {
        frontWheel.mount();
    }

    public void mountRearWheel() {
        rearWheel.mount();
    }
}
```

```
10. //----- MAIN-----
    public class Main {
        public static void main(String[] args) {
            Bicycle bike = new Bicycle();

            bike.stepOnPedals();
            bike.shiftUp();
```

```
bike.shiftUp();  
bike.applyRearBrake();  
  
bike.switchLightsOn();  
  
bike.removeRearWheel();  
bike.mountRearWheel();  
  
bike.switchLightsOff();  
}  
}
```